

MIC regression

Lucy Harrison

2026-03-17

Preamble

Fit regression models

Complete MIC data (NOS)

```
# apply log to the mics (mix of NOS and 90):
dat_nos_only <- dat_nos_only %>%
  mutate(mic_logged = log2(micnos_1))

# fit a model weighted by inverse sqrt sample sizes to determine residual errors:
# weights in full model
mod_init = lm(mic_logged ~ isolate_yr, dat_nos_only, weights = 1/sqrt(count))
summary(mod_init)

##
## Call:
## lm(formula = mic_logged ~ isolate_yr, data = dat_nos_only, weights = 1/sqrt(count))
##
## Weighted Residuals:
##      Min       1Q   Median       3Q      Max
## -3.00691 -0.38749 -0.00928  0.00416  2.43792
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -7.5494829  13.5260268  -0.558    0.578
## isolate_yr    0.0007906   0.0067526   0.117    0.907
##
## Residual standard error: 0.6178 on 159 degrees of freedom
## Multiple R-squared:  8.62e-05,    Adjusted R-squared:  -0.006203
## F-statistic: 0.01371 on 1 and 159 DF,  p-value: 0.9069

# setting weight to median absolute residuals
wts <- data.frame(isolate_yr = dat_nos_only$isolate_yr,
                  resid = mod_init$residuals) %>%
  group_by(isolate_yr) %>%
  summarise(wt = median(abs(resid)))

dat_nos_only <- dat_nos_only %>%
  left_join(wts)

## Joining with `by = join_by(isolate_yr)`
```

```
mod_weighted_nos_only <- lm(mic_logged ~ isolate_yr, dat_nos_only, weights = wt)
confint(mod_weighted_nos_only)
```

```
##                2.5 %      97.5 %
## (Intercept) -59.174330679 -3.82369886
## isolate_yr   -0.001150628  0.02654375
```

```
summary(mod_weighted_nos_only)
```

```
##
## Call:
## lm(formula = mic_logged ~ isolate_yr, data = dat_nos_only, weights = wt)
##
## Weighted Residuals:
##      Min       1Q   Median       3Q      Max
## -2.12998 -0.58710  0.00148  0.12213  2.94597
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -31.499015   14.012845  -2.248   0.026 *
## isolate_yr    0.012697    0.007011   1.811   0.072 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.8372 on 159 degrees of freedom
## Multiple R-squared:  0.02021,    Adjusted R-squared:  0.01405
## F-statistic: 3.279 on 1 and 159 DF,  p-value: 0.07205
```

```
# this is slope on log-scale:
```

```
coefficients(mod_weighted_nos_only)[2]
```

```
## isolate_yr
## 0.01269656
```

```
2*(coefficients(mod_weighted_nos_only)[2])
```

```
## isolate_yr
## 1.008839
```

```
2*confint(mod_weighted_nos_only)[2,]
```

```
##      2.5 %      97.5 %
## 0.9992028 1.0185690
```

All studies: MIC90

```
# apply log to the mics (mix of NOS and 90):
```

```
dat_agg <- dat_agg %>%
  mutate(mic_logged = log2(mic_90_agg))
```

```
# fit a model weighted by inverse sqrt sample sizes to determine residual errors:
# weights in full model
```

```
mod_init = lm(mic_logged ~ isolate_yr, dat_agg, weights = 1/sqrt(count))
summary(mod_init)
```

```
##
## Call:
```

```

## lm(formula = mic_logged ~ isolate_yr, data = dat_agg, weights = 1/sqrt(count))
##
## Weighted Residuals:
##      Min       1Q   Median       3Q      Max
## -0.7298 -0.2232  0.0002   0.1207   1.4036
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.141159   7.003181  -0.163   0.871
## isolate_yr  -0.002455   0.003500  -0.702   0.484
##
## Residual standard error: 0.3171 on 211 degrees of freedom
## Multiple R-squared:  0.002327, Adjusted R-squared:  -0.002401
## F-statistic: 0.4922 on 1 and 211 DF, p-value: 0.4837

# setting weight to median absolute residuals
wts <- data.frame(isolate_yr = dat_agg$isolate_yr,
                  resid = mod_init$residuals) %>%
  group_by(isolate_yr) %>%
  summarise(wt = median(abs(resid)))

dat_agg <- dat_agg %>%
  left_join(wts)

## Joining with `by = join_by(isolate_yr)`
mod_weighted_agg <- lm(mic_logged ~ isolate_yr, dat_agg, weights = wt)
confint(mod_weighted_agg)

##              2.5 %      97.5 %
## (Intercept) -28.94219706 20.13498260
## isolate_yr   -0.01305848  0.01149678

summary(mod_weighted_agg)

##
## Call:
## lm(formula = mic_logged ~ isolate_yr, data = dat_agg, weights = wt)
##
## Weighted Residuals:
##      Min       1Q   Median       3Q      Max
## -2.7873 -0.3961 -0.0023   0.0084   3.4116
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -4.4036072 12.4481080  -0.354   0.724
## isolate_yr  -0.0007808  0.0062283  -0.125   0.900
##
## Residual standard error: 0.7762 on 211 degrees of freedom
## Multiple R-squared:  7.449e-05, Adjusted R-squared:  -0.004664
## F-statistic: 0.01572 on 1 and 211 DF, p-value: 0.9003

# this is slope on log-scale:
coefficients(mod_weighted_agg)[2]

##      isolate_yr
## -0.0007808477

```

```
2**(coefficients(mod_weighted_agg)[2])
```

```
## isolate_yr  
## 0.9994589
```

```
2**confint(mod_weighted_agg)[2,]
```

```
## 2.5 % 97.5 %  
## 0.9909894 1.0080008
```

Now for some plots

Common attributes:

```
YLIM <- range(c(dat_agg$mic_90_agg, dat_nos_only$micnos_1))  
Y_LOWER <- log2(0.0018)  
SIZE_LIMS <- range(c(dat_agg$count, dat_nos_only$count))  
yticks <- c(0, 0.002, 0.004, 0.008, 0.016, 0.032, 0.064, 0.12)
```

Complete MIC data (NOS only)

```
X_LOWER <- 1965
```

here comes the model again:

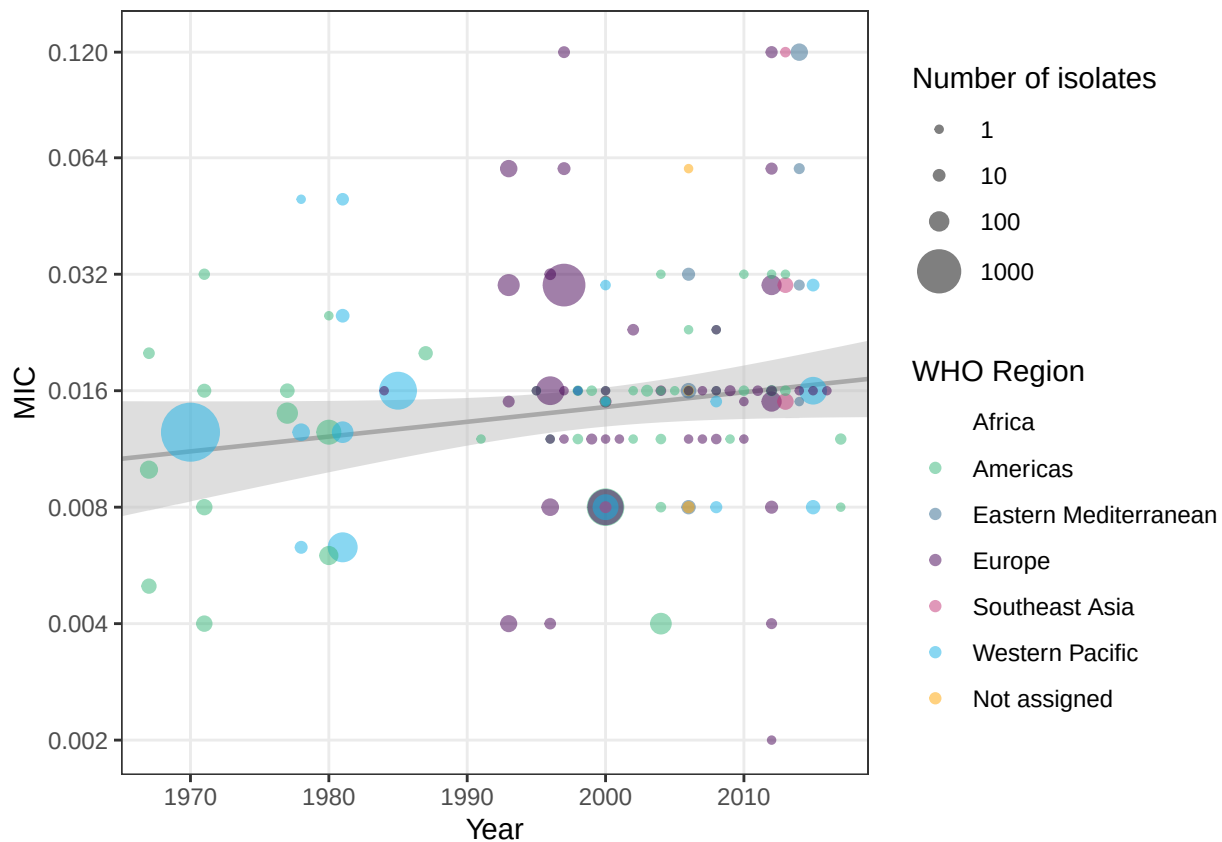
```
newdata <- data.frame(isolate_yr = seq(X_LOWER, max(dat_nos_only(isolate_yr) + 2))  
ci <- predict(mod_weighted_nos_only,  
              newdata = newdata,  
              interval = "confidence") %>%  
  as.data.frame() %>%  
  mutate(lwr = ifelse(lwr < Y_LOWER, Y_LOWER, lwr),  
         isolate_yr = seq(X_LOWER, max(dat_nos_only(isolate_yr) + 2))
```

```
p1 <- ggplot(dat_nos_only %>%  
  filter(!is.na(micnos_1)) %>%  
  group_by(isolate_yr, micnos_1, who_name) %>%  
  summarise(count = sum(count)) %>%  
  mutate(who_name = factor(who_name, levels = c("Africa",  
                                                "Americas",  
                                                "Eastern Mediterranean",  
                                                "Europe",  
                                                "Southeast Asia",  
                                                "Western Pacific",  
                                                "Not assigned")))) %>%  
  arrange(desc(count))) +  
  geom_ribbon(data = ci, aes(ymin = 2**lwr, ymax = 2**upr, x = isolate_yr),  
            alpha = 0.5, fill = "grey", colour = NA) +  
  geom_line(data = ci, aes(x = isolate_yr, y = 2**fit),  
           colour = "darkgrey", linewidth = 0.8) +  
  geom_point(aes(x = isolate_yr, y = micnos_1,  
                col = who_name,  
                size = count), alpha = 0.5) +  
  scale_color_discrete(name = "WHO Region", type = pal, drop = FALSE) +  
  scale_size_continuous(range = c(1,10), name = "Number of isolates",  
                       breaks = c(1, 10, 100, 1000), limits = SIZE_LIMS) +  
  scale_y_continuous(breaks = yticks, minor_breaks = NULL, name = "MIC") +
```

```
scale_x_continuous(limits = c(X_LOWER, 2017 + 2), expand = c(0,0),
                   breaks = seq(1970, 2010, 10), minor_breaks = NULL) +
labs(x = "Year", ylab = "MIC") +
coord_trans(y = "log2", ylim = YLIM) +
theme_bw()
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by isolate_yr, micnos_1, and who_name.
## i Output is grouped by isolate_yr and micnos_1.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(isolate_yr, micnos_1, who_name))` for per-operation
## grouping (`?dplyr::dplyr_by`) instead.
```

p1



All studies: MIC90

```
X_LOWER <- 1927

# here comes the model again:
newdata <- data.frame(isolate_yr = seq(X_LOWER, max(dat_agg$isolate_yr) + 2))
ci <- predict(mod_weighted_agg,
              newdata = newdata,
              interval = "confidence") %>%
as.data.frame() %>%
mutate(lwr = ifelse(lwr < Y_LOWER, Y_LOWER, lwr),
       isolate_yr = seq(X_LOWER, max(dat_agg$isolate_yr) + 2))
```

```

p2 <- ggplot(dat_agg %>%
  filter(!is.na(mic_90_agg)) %>%
  group_by(isolate_yr, mic_90_agg, who_name) %>%
  summarise(count = sum(count)) %>%
  mutate(who_name = factor(who_name, levels = c("Africa",
                                                "Americas",
                                                "Eastern Mediterranean",
                                                "Europe",
                                                "Southeast Asia",
                                                "Western Pacific",
                                                "Not assigned")))) %>%

  arrange(desc(count))) +
geom_ribbon(data = ci, aes(ymin = 2**lwr, ymax = 2**upr, x = isolate_yr),
  alpha = 0.5, fill = "grey", colour = NA) +
geom_line(data = ci, aes(x = isolate_yr, y = 2**fit),
  colour = "darkgrey", linewidth = 0.8) +
geom_point(aes(x = isolate_yr, y = mic_90_agg,
  col = who_name,
  size = count), alpha = 0.5) +
scale_color_discrete(name = "WHO Region",
  type = pal,
  guide = guide_legend(override.aes = list(size = 5),
    ncol = 2)) +
scale_size_continuous(range = c(1,10), name = "Number of isolates",
  breaks = c(1, 10, 100, 1000), limits = SIZE_LIMS) +
scale_y_continuous(breaks = yticks, minor_breaks = NULL) +
scale_x_continuous(limits = c(X_LOWER, 2017 + 2), expand = c(0,0),
  breaks = seq(1930, 2010, 10), minor_breaks = NULL) +
xlab("Year") +
ylab("MIC90") +
coord_trans(y = "log2", ylim = YLIM) +
theme_bw() +
theme(
  legend.box = "horizontal",
  legend.box.just = "top",
  plot.background = element_rect(fill = "transparent", colour = "transparent")
)

```

```

## `summarise()` has regrouped the output.
## i Summaries were computed grouped by isolate_yr, mic_90_agg, and who_name.
## i Output is grouped by isolate_yr and mic_90_agg.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(isolate_yr, mic_90_agg, who_name))` for per-operation
##   grouping (`?dplyr::dplyr_by`) instead.

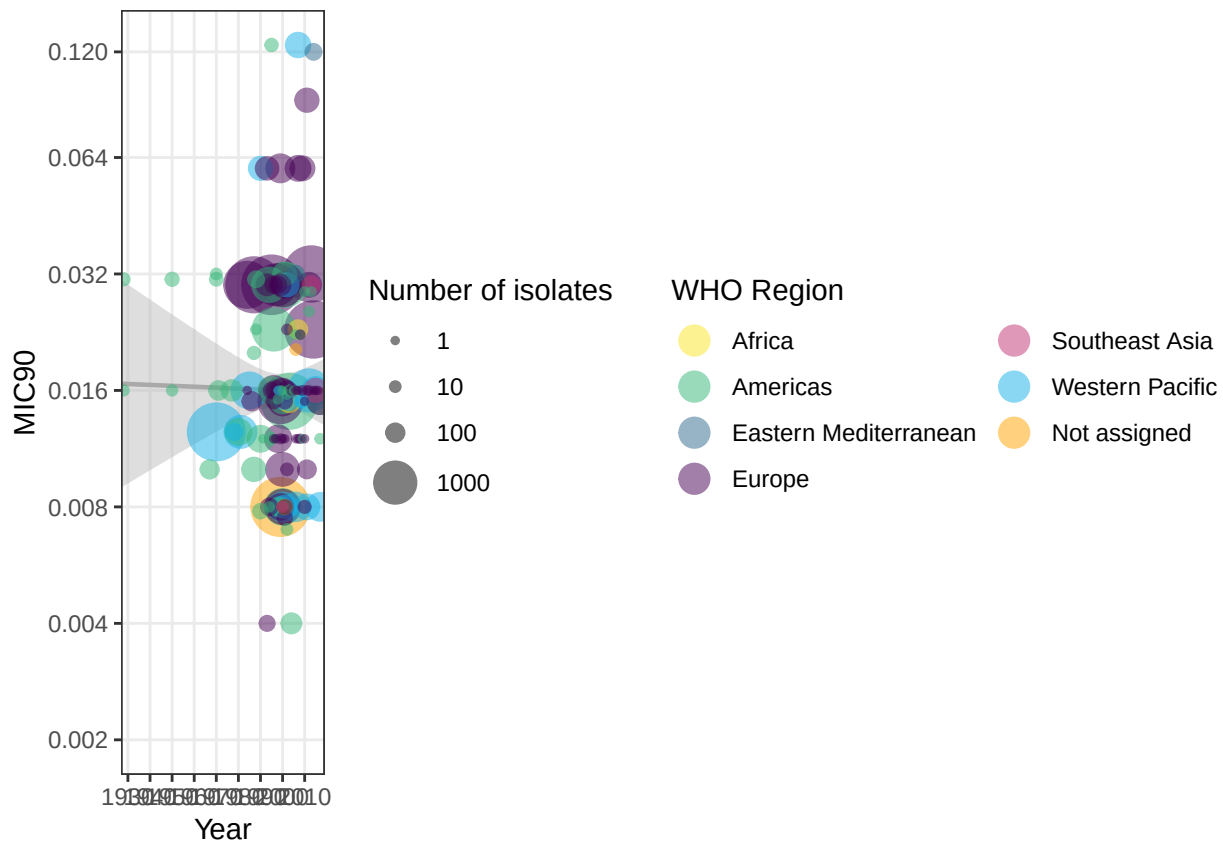
```

p2

```

## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_point()`).

```



Putting it all together:

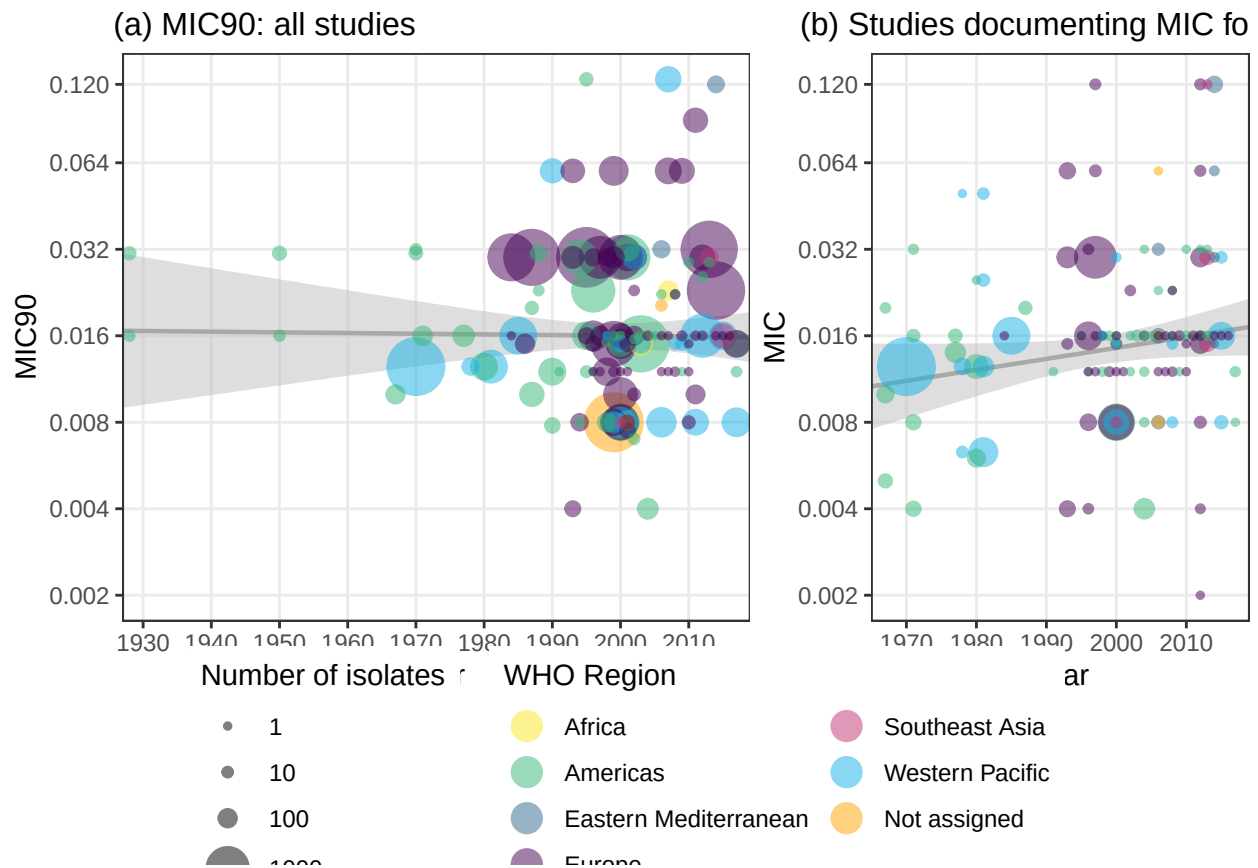
```
p <- plot_grid(p2 + theme(legend.position = "none",
  plot.margin = margin(t = 20, r = 0, b = 0, l = 5, unit = "pt")),
  p1 + theme(legend.position = "none",
  plot.margin = margin(t = 20, r = 0, b = 0, l = 5, unit = "pt")),
  labels = c("(a) MIC90: all studies",
    "(b) Studies documenting MIC for all isolates"),
  label_fontface = "plain",
  rel_widths = c(0.6, 0.4),
  label_x = c(0.15, 0.1), hjust = 0, label_size = 12)
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_point()`).
```

```
plot_grid(p, get_legend(p2), ncol = 1, rel_heights = c(0.8, 0.2))
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_point()`).
```

```
## Warning in get_plot_component(plot, "guide-box"): Multiple components found;
## returning the first one. To return all, use `return_all = TRUE`.
```



```
ggsave("figs/MIC_regression_twice.png", height = 7, width = 9, bg = "transparent")
ggsave("figs/MIC_regression_twice.svg", height = 7, width = 9, bg = "transparent")
```

TODO - report Rsq

Total isolates by MIC90/raw MIC

Figure 3: what are the spreads of the two subsets?

```
hist_dat <- dat_raw %>% # from Rhys' aggregate file
  filter(!is.na(mic90_1)) %>%
  bind_rows(dat_nos_only) %>% # NOS only
  pivot_longer(cols = c(mic90_1, mic90_2), names_to = "type") %>%
  filter(!is.na(value)) %>%
  mutate(value = round(value, digits = 4)) %>%
  summarise(count = sum(count), .by = c(value, type)) %>%
  mutate(logmic = log2(value)) %>%
  mutate(type = ifelse(type == "mic90_1",
    "Aggregate MIC90 reported only",
    "Non-aggregated MIC reported"))

x_breaks = c(0.002, 0.004, 0.008, 0.016, 0.032, 0.064, 0.12)

# for labels on bars:
to_lab <- hist_dat %>%
  # only highest ones:
  filter((type == "Aggregate MIC90 reported only" & count > 2000) |
```



```

      (type == "Non-aggregated MIC reported" & count > 1000)) %>%
mutate(nudge_yy = ifelse(type == "Non-aggregated MIC reported",
                        1200, 300),
      hjustt = ifelse(count %in% c(2727, 2100, 3097), 0, 0.5),
      vjustt = ifelse(count %in% c(2727, 3097, 2030), 0.5, 1),
      #nudge_x = ifelse(count %in% c(2727, 2100), 0, 0.5),
      lab = ifelse(count %in% c(2727, 3097, 2030),
                    paste0("MIC:\n", value, "\n(", count, ")"),
                    paste0("MIC: ", value, "\n(", count, ")")))

xlim = c(0.0019, 0.15)

p1 <- ggplot(data = hist_dat %>%
             filter(type == "Aggregate MIC90 reported only")) +
  geom_bar(aes(x = logmic, y = count,
              fill = type),
           alpha = 0.6,
           stat = "identity",
           width = 0.05) +
  # here are labels on top of higher bars:
  geom_text(aes(x = logmic, y = count, label = lab,
               hjust = hjustt),
            data = to_lab %>%
              filter(type == "Aggregate MIC90 reported only"),
            nudge_y = 1500,
            vjust = 1,
            size = 2.5) +
  facet_wrap(~ type, ncol = 1, scales = "free_y") +
  scale_x_continuous(breaks = log2(x_breaks),
                    labels = x_breaks,
                    limits = log2(xlim)) +
  scale_fill_manual(values = pal[2]) +
  ylab("") +
  xlab("MIC90") +
  theme_bw() +
  theme(legend.position = "none")

p2 <- ggplot(data = hist_dat %>%
             filter(type == "Non-aggregated MIC reported")) +
  geom_bar(aes(x = logmic, y = count,
              fill = type),
           alpha = 0.6,
           stat = "identity",
           width = 0.05) +
  # here are labels on top of higher bars:
  geom_text(aes(x = logmic, y = count, label = lab),
            data = to_lab %>%
              filter(type == "Non-aggregated MIC reported"),
            nudge_y = 400,
            vjust = 1,
            size = 2.5) +
  facet_wrap(~ type, ncol = 1, scales = "free_y") +
  scale_x_continuous(breaks = log2(x_breaks),

```

```

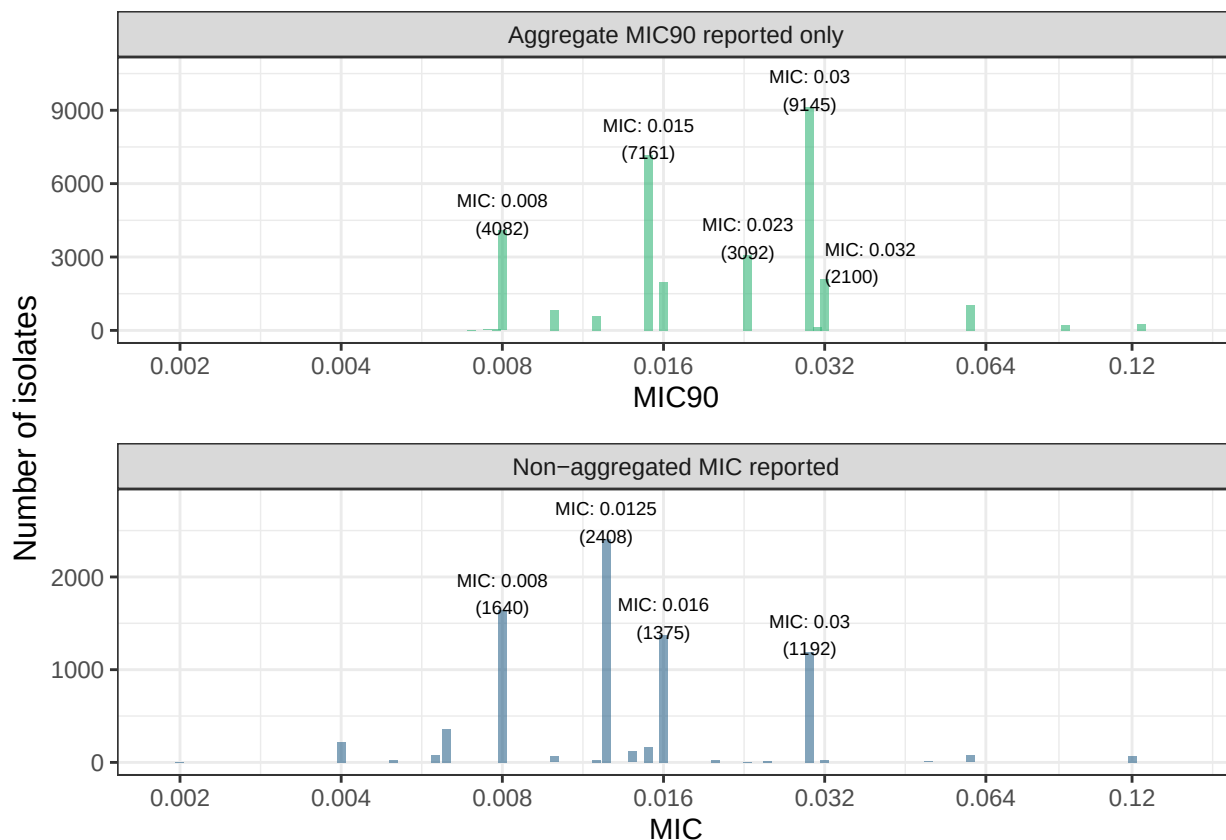
        labels = x_breaks,
        limits = log2(xlim)) +
scale_fill_manual(values = pal[3]) +
ylab("") +
xlab("MIC") +
theme_bw() +
theme(legend.position = "none")

p <- plot_grid(p1, p2, ncol = 1)

## Warning: `position_stack()` requires non-overlapping x intervals.

ggdraw(p) +
  draw_label("Number of isolates", x = 0, y = 0.5, angle = 90,
            vjust = 1.5, size = 12)

```



```

ggsave("figs/all_mics.png", height = 6, width = 6)
ggsave("figs/all_mics.svg", height = 6, width = 6)

# ggplot(data = hist_dat) +
#   geom_bar(aes(x = logmic, y = count,
#               fill = type),
#           alpha = 0.6,
#           stat = "identity",
#           width = 0.05) +
#   # here are labels on top of higher bars:
#   geom_text(aes(x = logmic, y = count, label = lab,
#               hjust = hjustt, vjust = vjustt),

```

```
#           data = to_lab,
#           nudge_y = to_lab$nudge_yy,
#           size = 2.5) +
# # abandoning
# facet_wrap(~ type, ncol = 1, scales = "free_y") +
# scale_x_continuous(breaks = log2(x_breaks),
#                    labels = x_breaks,
#                    limits = log2(xlim)) +
# scale_fill_manual(values = pal[2:3]) +
# ylab("") +
# xlab("MIC90") +
# theme_bw() +
# theme(legend.position = "none")
```

Ordinal regression

What happens when we try to reapply the linear regression treating MICs as ordinal? We could really treat MICs as categories as intermediate MICs are not (usually) tested. For this reason, we've had a look at ordinal regression in the chunk below. We resolved not to do this because the data was all created under different settings with (probably) different dilutions, different methods of calculating MIC90, etc.

First let's have a look at raw MICs only:

```
library(MASS)
```

```
##
```

```
## Attaching package: 'MASS'
```

```
## The following object is masked from 'package:dplyr':
```

```
##
```

```
## select
```

```
ordinal_regn_please <- function(dat, bins, ylab, main,
                                weight = TRUE, xlower = 1927){
```

```
  dat <- mutate(dat,
                mic_binned = cut(mic,
                                breaks = c(0, bins),
                                ordered_result = TRUE))
```

```
  # two poss models:
```

```
  if (weight){
```

```
    model <- polr(mic_binned ~ isolate_yr,
                  data = dat,
                  Hess = TRUE,
                  weights = count, # here, weights are frequencies
                  method = "probit") # fails to converge under logit: fatter tail
```

```
  } else {
```

```
    model <- polr(mic_binned ~ isolate_yr,
                  data = dat,
                  Hess = TRUE)
```

```
  }
```

```
  # Fit the model - no need to log: that would be for visualisation purposes only under ordinal model
  # tried using wts from logged model ..... not thinking too hard about that but we should at some
  # it looks like weights need to be integers as we end up assuming binomial glm for ordinal *logistic*
  # haven't thought too hard about this ... may need to be probit link
```

```

print(summary(model))
# note the slope here

new_data = data.frame(isolate_yr = seq(xlower,
                                       max(dat$isolate_yr) + 2))

preds <- predict(model,
                 newdata = new_data,
                 type = "probs") %>%
  as.data.frame() %>%
  mutate(max_probs = max.col(across(starts_with("("))) %>%
  mutate(isolate_yr = new_data$isolate_yr)

if (length(unique(preds$max_probs)) == 1){
  message(paste0("Most probable class stays the same:",
                unique(preds$max_probs)))
} else {
  message(paste0(c("Most probable class changes:\n", unique(preds$max_probs)),
                collapse = " "))
}

preds_long <- pivot_longer(preds, starts_with("(") %>%
  mutate(lower = str_extract(name, "(?<=\\()[^,]+") %>% as.numeric(),
         upper = str_extract(name, "(?<=,)[^]]+") %>% as.numeric(),
         height = lower + (upper - lower)*value) %>%
  filter(lower != 0)

preds_change <- preds %>%
  group_by(max_probs) %>%
  summarise(xmin = first(isolate_yr) - 0.5,
            xmax = last(isolate_yr) + 0.5) %>%
  mutate(ymin = bins[max_probs - 1],
         ymax = bins[max_probs])

# so now we have colour bars representing class probability between the lower
# and upper extents of the bins ... which don't change much because there isn't
# much change in MIC slope over time
ggplot(data = dat) +
  geom_point(aes(x = isolate_yr, y = mic, size = count),
            alpha = 0.5, col = "grey30") +
  scale_y_continuous(trans = "log2", breaks = bins, minor_breaks = NULL) +
  geom_ribbon(data = preds_long,
            aes(ymin = lower, ymax = height, fill = name, x = isolate_yr),
            alpha = 0.5) +
  scale_fill_discrete("MIC interval") +
  scale_size_continuous(range = c(1,10), name = "Number of isolates",
                        breaks = c(1, 10, 100, 1000), limits = SIZE_LIMS) +
  # (the -1 is because I removed the bottom bin)
  # this is a bit gross but I've put the line in the middle
  # geom_hline(yintercept = 2*(sum(log2(bins[preds$max_probs[1] + c(0,-1)])) / 2)) +
  # annotate(geom = "rect",
  #         xmin = xlower,
  #         xmax = max(new_data$isolate_yr),

```

```

#         ymin = bins[preds$max_probs[1] - 1],
#         ymax = bins[preds$max_probs[1]], col = "black", fill = NA) +
geom_rect(data = preds_change,
          aes(xmin = xmin, xmax = xmax, ymin = ymin, ymax = ymax),
          col = "black", fill = NA) +
xlab("Year") +
ylab(ylab) +
labs(title = main) +
theme_bw() +
theme(
  legend.box = "horizontal",
  legend.box.just = "top",
  plot.background = element_rect(fill = "transparent", colour = "transparent"),
  plot.title = element_text(size=11)
)
}

p1 <- ordinal_regn_please(dat_nos_only %>% mutate(mic = micnos_1),
  bins = c(0.002, 0.004, 0.008, 0.016, 0.032, 0.064, 0.128),
  xlower = 1965, ylab = "MIC",
  main = "(b) Studies documenting MIC for all isolates")

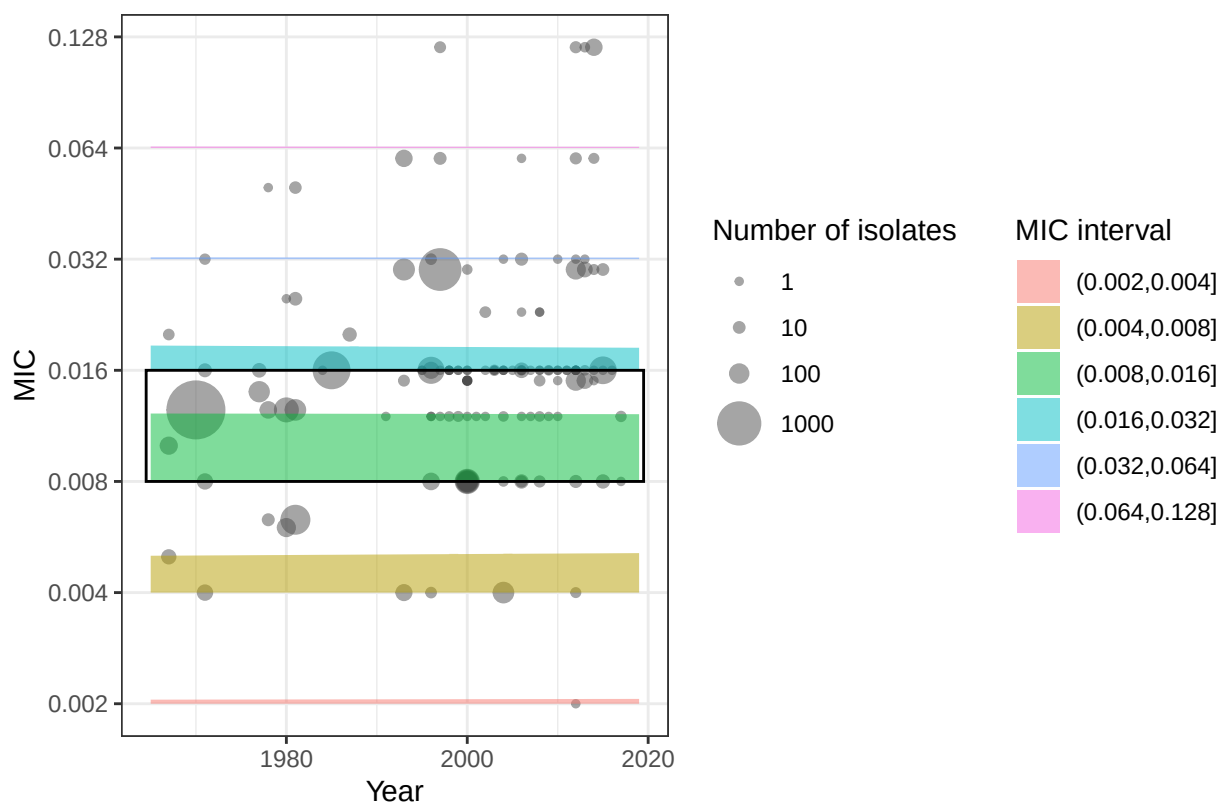
```

```

## Call:
## polr(formula = mic_binned ~ isolate_yr, data = dat, weights = count,
##       Hess = TRUE, method = "probit")
##
## Coefficients:
##               Value Std. Error t value
## isolate_yr -0.001373  1.493e-05  -91.96
##
## Intercepts:
##               Value      Std. Error  t value
## (0,0.002] |(0.002,0.004]    -6.3880      0.0006 -10773.9635
## (0.002,0.004] |(0.004,0.008]    -4.6423      0.0027  -1734.4102
## (0.004,0.008] |(0.008,0.016]    -3.2684      0.0301  -108.4971
## (0.008,0.016] |(0.016,0.032]    -1.8135      0.0332   -54.6335
## (0.016,0.032] |(0.032,0.064]    -0.6690      0.0430  -15.5468
## (0.032,0.064] |(0.064,0.128]    -0.3559      0.0432   -8.2430
##
## Residual Deviance: 18571.14
## AIC: 18585.14
## Most probable class stays the same:4
p1

```

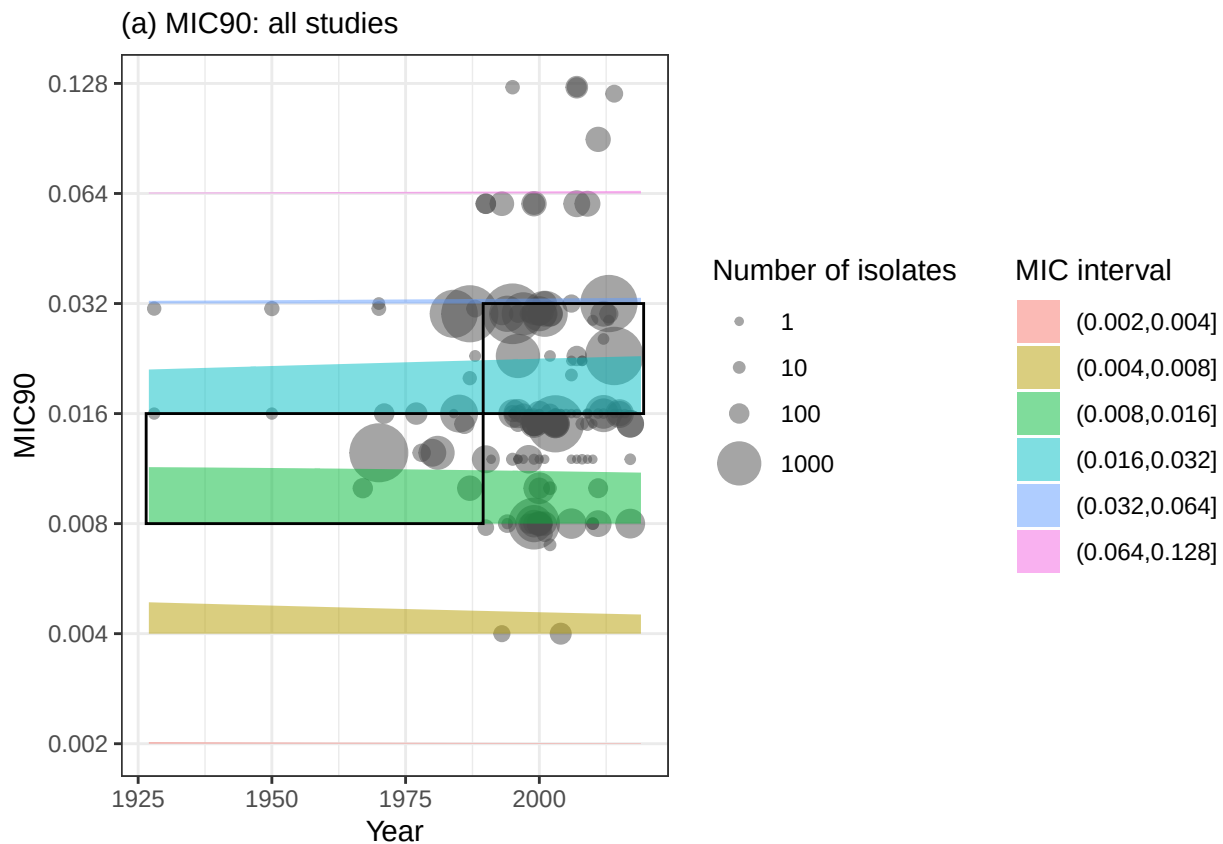
(b) Studies documenting MIC for all isolates



```
p2 <- ordinal_regn_please(dat_agg %>% mutate(mic = mic_90_agg),
  bins = c(0.002, 0.004, 0.008, 0.016, 0.032, 0.064, 0.128),
  ylab = "MIC90", main = "(a) MIC90: all studies")
```

```
## Call:
## polr(formula = mic_binned ~ isolate_yr, data = dat, weights = count,
##       Hess = TRUE, method = "probit")
##
## Coefficients:
##               Value Std. Error t value
## isolate_yr 0.004084 1.307e-05   312.4
##
## Intercepts:
##               Value      Std. Error t value
## (0,0.002] |(0.002,0.004]    3.6998     0.0006 6032.5293
## (0.002,0.004] |(0.004,0.008]    5.5640     0.0028 1995.9977
## (0.004,0.008] |(0.008,0.016]    7.1288     0.0263  271.3897
## (0.008,0.016] |(0.016,0.032]    8.2732     0.0265  311.8468
## (0.016,0.032] |(0.032,0.064]    9.8581     0.0289  341.3249
## (0.032,0.064] |(0.064,0.128]   10.3756     0.0290  357.6436
##
## Residual Deviance: 93146.55
## AIC: 93160.55
##
## Most probable class changes:
##    4 5
```

p2



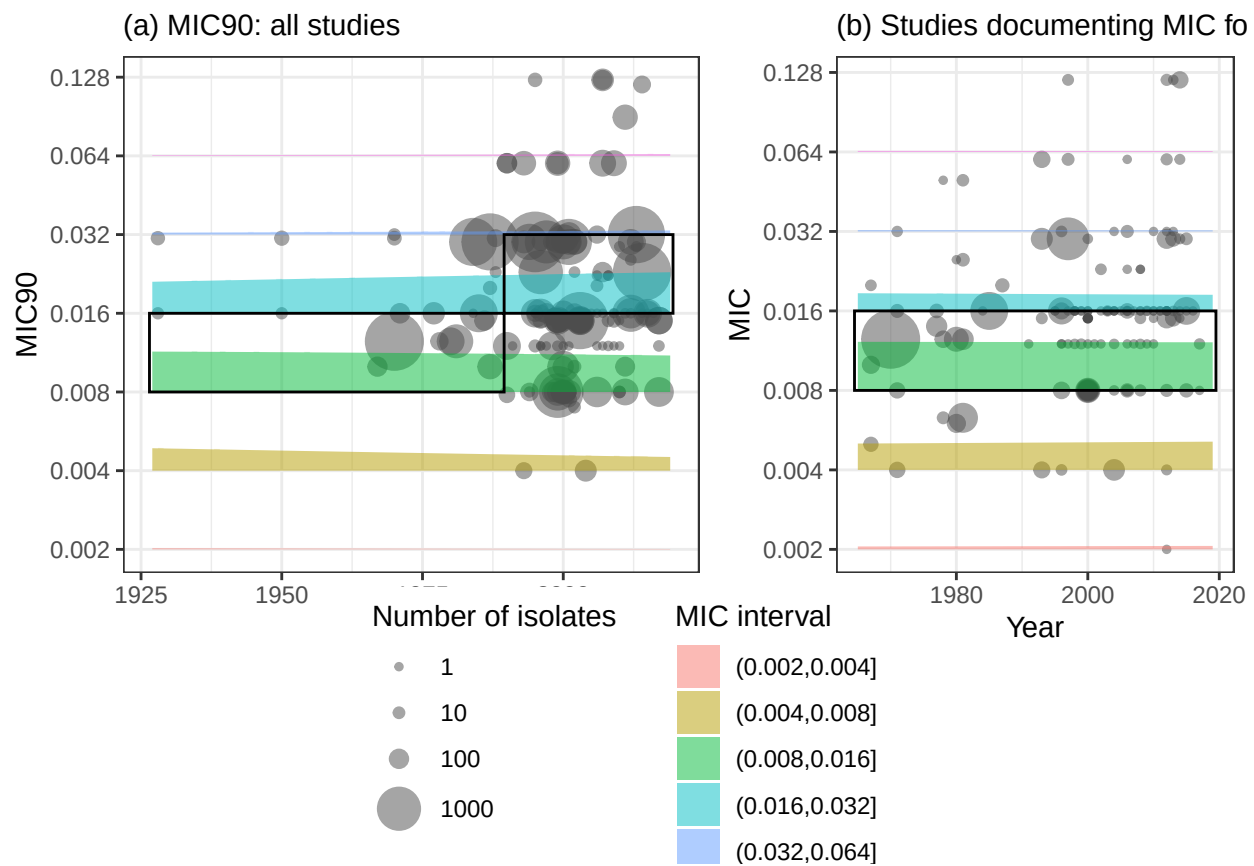
Caption: Ordinal regression of MIC data (as in Figure 4): coloured regions represent predicted probability of MIC interval over time; the most probable interval, which does not change over time in either case, is highlighted with a black rectangle. Intervals are right-inclusive: the interval (0.008, 0.016] includes MIC 0.016 and excludes MIC 0.008.

Here we put the subplots together:

```
p <- plot_grid(p2 + theme(legend.position = "none"),
  p1 + theme(legend.position = "none"),
  rel_widths = c(0.6, 0.45))

plot_grid(p, get_legend(p2), ncol = 1, rel_heights = c(0.8, 0.25))
```

```
## Warning in get_plot_component(plot, "guide-box"): Multiple components found;
## returning the first one. To return all, use `return_all = TRUE`.
```



```
ggsave("figs/ordinal_regression.png", height = 7, width = 9, bg = "transparent")
ggsave("figs/ordinal_regression.svg", height = 7, width = 9, bg = "transparent")
```